

1. Problem Statement:

A software company is developing a user registration system. Before creating an account, the system must validate user credentials based on predefined security policies.

Develop a Python application that validates:

Email Address

Password

Mobile Number

using Regular Expressions.

Validation Rules

Email

Must start with an alphabet.

May contain letters, digits, '.', '_' before '@'.

Domain should contain only alphabets.

Valid extensions: .com, .org, .edu, .net, .in

Password

Minimum 8 characters

At least one uppercase letter

At least one lowercase letter

At least one digit

At least one special character (@, #, \$, %, &, !)

Mobile Number

Must contain exactly 10 digits.

First digit should be between 6–9.

Expected Output

Display:

Valid Email / Invalid Email

Strong Password / Weak Password

Valid Mobile Number / Invalid Mobile Number

Code

```
import re

email = input("Enter Email: ")

password = input("Enter Password: ")

mobile = input("Enter Mobile Number: ")

email_pattern = r'^[A-Za-z][A-Za-z0-9._]*@[A-Za-z]+\.(com|org|edu|net|in)$'

password_pattern = r'^(?=.*[A-Z])(?=.*[a-z])(?=.*[0-9])(?=.*[@#$%&!]).{8,}$'

mobile_pattern = r'^[6-9][0-9]{9}$'

if re.match(email_pattern, email):

    print("Valid Email")

else:

    print("Invalid Email")

if re.match(password_pattern, password):

    print("Strong Password")

else:

    print("Weak Password")

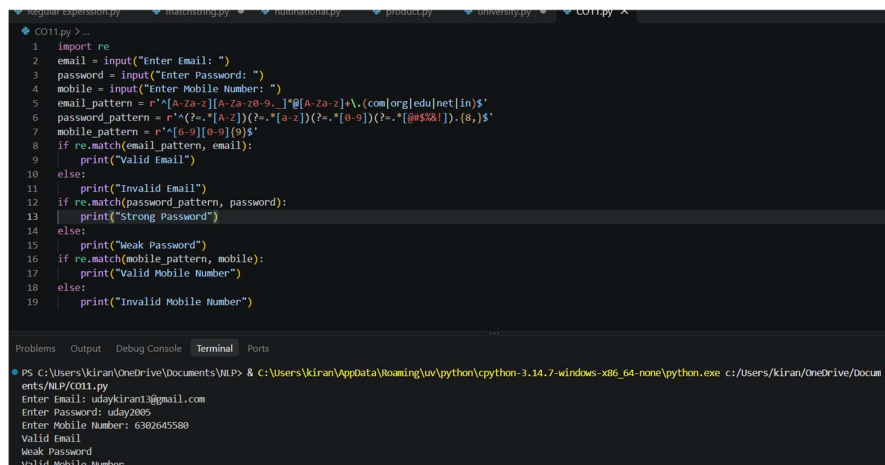
if re.match(mobile_pattern, mobile):

    print("Valid Mobile Number")

else:

    print("Invalid Mobile Number")
```

Output



```
CO11.py > ...
1  import re
2  email = input("Enter Email: ")
3  password = input("Enter Password: ")
4  mobile = input("Enter Mobile Number: ")
5  email_pattern = r'^[A-Za-z][A-Za-z0-9._]*@[A-Za-z]+\.(com|org|edu|net|in)$'
6  password_pattern = r'^(?=.*[A-Z])(?=.*[a-z])(?=.*[0-9])(?=.*[@#$%&!]).{8,}$'
7  mobile_pattern = r'^[6-9][0-9]{9}$'
8  if re.match(email_pattern, email):
9      print("Valid Email")
10 else:
11     print("Invalid Email")
12 if re.match(password_pattern, password):
13     print("Strong Password")
14 else:
15     print("Weak Password")
16 if re.match(mobile_pattern, mobile):
17     print("Valid Mobile Number")
18 else:
19     print("Invalid Mobile Number")

Problems  Output  Debug Console  Terminal  Ports
PS C:\Users\kiran\OneDrive\Documents\MLP> & C:\Users\kiran\AppData\Local\Programs\Python\Python314\python.exe c:\Users\kiran\OneDrive\Documents\MLP\CO11.py
Enter Email: udaykiran@gmail.com
Enter Password: uday2005
Enter Mobile Number: 6302645580
Valid Email
Valid Password
Valid Mobile Number
```

2. Problem Statement

Develop a Python-based Deterministic Finite Automata (DFA) simulator.

The program should:

- Accept the DFA description
 - States
 - Input alphabet
 - Transition table
 - Initial state
 - Final state(s)
- Accept multiple input strings.
- Simulate state transitions.
- Display whether the string is Accepted or Rejected.

Example

Language:

Strings ending with "ab"

Input

abaab

Output

Transition Path:

$q_0 \rightarrow q_1 \rightarrow q_0 \rightarrow q_1 \rightarrow q_2$

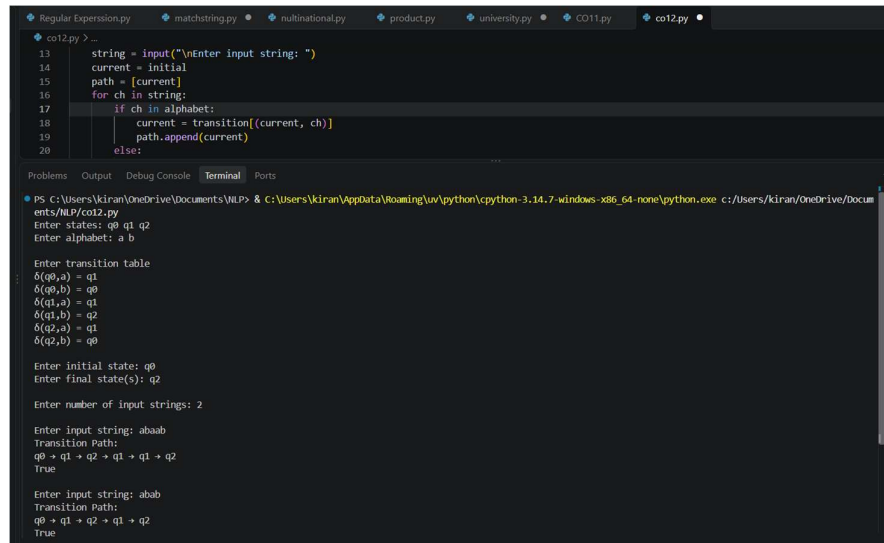
Accepted

Code

```
states = input("Enter states: ").split()
alphabet = input("Enter alphabet: ").split()
print("\nEnter transition table")
transition = {}
for state in states:
    for symbol in alphabet:
```

```
next_state = input("δ(" + state + "," + symbol + ") = ")
transition[(state, symbol)] = next_state
initial = input("\nEnter initial state: ")
final = input("Enter final state(s): ").split()
n = int(input("\nEnter number of input strings: "))
for i in range(n):
    string = input("\nEnter input string: ")
    current = initial
    path = [current]
    for ch in string:
        if ch in alphabet:
            current = transition[(current, ch)]
            path.append(current)
        else:
            current = "Invalid"
            break
    print("Transition Path:")
    print(" → ".join(path))
    if current in final:
        print("True")
    else:
        print("False")
```

Output



```
co12.py > ...
13 string = input("\nEnter input string: ")
14 current = initial
15 path = [current]
16 for ch in string:
17     if ch in alphabet:
18         current = transition[(current, ch)]
19         path.append(current)
20     else:
21         break
22
23 # Print the path
24 print("Transition Path: ", path)
25
26 # Check if the path is valid
27 if path[-1] == final:
28     print("True")
29 else:
30     print("False")
```

PS C:\Users\Kiran\OneDrive\Documents\NLP> & C:\Users\Kiran\AppData\Local\Programs\Python\Python314\python.exe c:/Users/Kiran/OneDrive/Docu
ents/NLP/co12.py
Enter states: q0 q1 q2
Enter alphabet: a b

Enter transition table
delta(q0,a) = q1
delta(q0,b) = q0
delta(q1,a) = q1
delta(q1,b) = q2
delta(q2,a) = q1
delta(q2,b) = q0

Enter initial state: q0
Enter final state(s): q2

Enter number of input strings: 2

Enter input string: abaab
Transition Path:
q0 -> q1 -> q2 -> q1 -> q1 -> q2
True

Enter input string: abab
Transition Path:
q0 -> q1 -> q2 -> q1 -> q2
True

3. Problem Statement

Develop a Python-based text search engine that performs pattern matching using Regular Expressions.

The system should support:

Word Search

Prefix Search

Suffix Search

Date Search

Phone Number Search

Hashtag Search

Mention (@username) Search

Example Input

Meeting on 12/09/2026

Call 9876543210

#NLP

@OpenAI

natural language processing

User Menu

1. Search Date
2. Search Phone Number
3. Search Hashtag
4. Search Mention
5. Search Prefix
6. Search Suffix

Expected Output

Display all matching patterns based on user selection.

Code

```
import re

text = input("Enter text: ")

print("\nUser Menu")

print("1. Search Date")

print("2. Search Phone Number")

print("3. Search Hashtag")

print("4. Search Mention")

print("5. Search Prefix")

print("6. Search Suffix")

choice = input("Enter your choice: ")

if choice == "1":

    pattern = r'\b\d{2}/\d{2}/\d{4}\b'

    result = re.findall(pattern, text)

    print("Dates:", result)

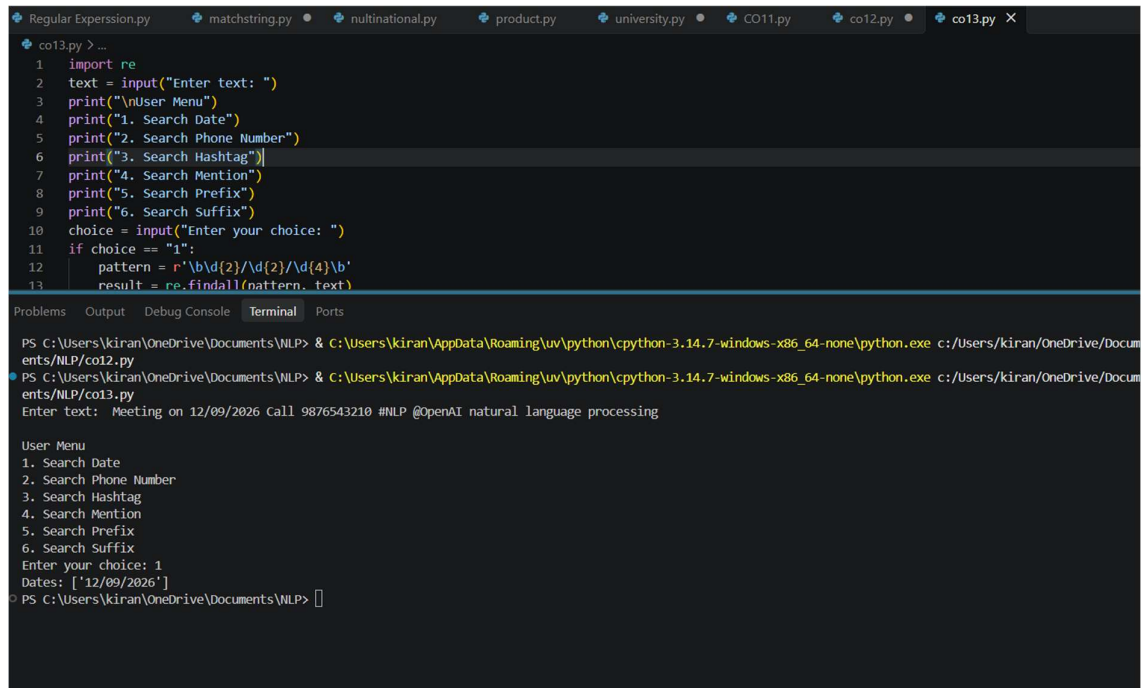
elif choice == "2":

    pattern = r'\b[6-9]\d{9}\b'

    result = re.findall(pattern, text)
```

```
    print("Phone Numbers:", result)
elif choice == "3":
    pattern = r'#\w+'
    result = re.findall(pattern, text)
    print("Hashtags:", result)
elif choice == "4":
    pattern = r'@\w+'
    result = re.findall(pattern, text)
    print("Mentions:", result)
elif choice == "5":
    word = input("Enter prefix: ")
    pattern = r'\b' + re.escape(word) + r'\w*'
    result = re.findall(pattern, text, re.IGNORECASE)
    print("Prefix Matches:", result)
elif choice == "6":
    word = input("Enter suffix: ")
    pattern = r'\b\w*' + re.escape(word) + r'\b'
    result = re.findall(pattern, text, re.IGNORECASE)
    print("Suffix Matches:", result)
else:
    print("Invalid Choice")
```

Output



The image shows a Visual Studio Code editor window with several tabs open. The active tab is 'co13.py'. The code in the editor is a Python script that uses regular expressions to search for dates in a given text. The script prompts the user to enter text and then displays a menu of search options. The terminal output shows the script being executed, with the user entering the text 'Meeting on 12/09/2026 Call 9876543210 #NLP @OpenAI natural language processing'. The script then displays the search menu and the user's choice of '1'. The terminal output shows the dates found in the text: ['12/09/2026'].

```
co13.py > ...
1 import re
2 text = input("Enter text: ")
3 print("\nUser Menu")
4 print("1. Search Date")
5 print("2. Search Phone Number")
6 print("3. Search Hashtag")
7 print("4. Search Mention")
8 print("5. Search Prefix")
9 print("6. Search Suffix")
10 choice = input("Enter your choice: ")
11 if choice == "1":
12     pattern = r'\b\d{2}/\d{2}/\d{4}\b'
13     result = re.findall(pattern, text)
```

Problems Output Debug Console Terminal Ports

PS C:\Users\kiran\OneDrive\Documents\NLP> & C:\Users\kiran\AppData\Roaming\uv\python\cpython-3.14.7-windows-x86_64-none\python.exe c:/Users/kiran/OneDrive/Docum
ents/NLP/co12.py
PS C:\Users\kiran\OneDrive\Documents\NLP> & C:\Users\kiran\AppData\Roaming\uv\python\cpython-3.14.7-windows-x86_64-none\python.exe c:/Users/kiran/OneDrive/Docum
ents/NLP/co13.py
Enter text: Meeting on 12/09/2026 Call 9876543210 #NLP @OpenAI natural language processing

User Menu
1. Search Date
2. Search Phone Number
3. Search Hashtag
4. Search Mention
5. Search Prefix
6. Search Suffix
Enter your choice: 1
Dates: ['12/09/2026']
PS C:\Users\kiran\OneDrive\Documents\NLP>